

Phase 1 Recap & Mini System

Phase 1 · Fundamentals · Estimated time: 1.5 hours

1. Concept From First Principles: Putting It All Together

If any single chapter in Phase 1 felt shaky while reading this recap, that's useful signal, not failure — go back to that specific chapter before Phase 2, since Phase 2 assumes every Phase 1 mental model is already load-bearing. Treat this chapter as a diagnostic as much as a summary.

This chapter has no new concepts. Its job is to force you to combine everything from Chapters 1–14 into one coherent design, the way an actual interview would — because knowing 14 isolated concepts is not the same skill as combining them under time pressure. We'll design a **Pastebin-style service** (users paste text, get back a short shareable link) — simple enough to fully solve at Phase 1 level, but rich enough to touch nearly every chapter.

Step 1 — Clarify requirements (functional + non-functional)

- **Functional:** a user can paste text and get a short URL back; anyone with the URL can view the paste; pastes can optionally expire.
- **Non-functional:** reads (viewing a paste) will vastly outnumber writes (creating one); the service should feel instant; it should survive a single server failing (Chapter 12, 13).

Step 2 — Back-of-the-envelope estimation (Chapter 14)

Assume 1 million new pastes/day, and a generous 20:1 read:write ratio (pastes get shared and viewed many times). Write QPS $\approx 1,000,000 / 100,000 \approx 10/\text{sec}$. Read QPS $\approx 200/\text{sec}$. Average paste size $\approx 2\text{KB}$, so daily storage $\approx 2\text{GB/day}$, $\sim 730\text{GB/year}$ — entirely manageable on a single well-run relational database for a long time, which immediately tells us we don't need sharding at this stage (Phase 2 territory, deliberately not needed yet).

Step 3 — High-level architecture (Chapters 2, 6, 8, 9, 10, 11)

Client → DNS (Chapter 3) resolves our domain → Load Balancer (Chapter 9) → a pool of stateless application servers (Chapter 12) → a relational database (Chapter 6) storing paste metadata and short-code mappings. Because reads dominate 20:1, we put a cache (Chapter 8, cache-aside) in front of the database for paste lookups — the single highest-leverage addition given our access pattern. If pastes can include large attachments, those go to object storage (Chapter 11), not the database. Static assets (the web UI itself) are served via CDN (Chapter 10).

Step 4 — The core algorithm: generating short codes

Each paste needs a short, unique identifier (e.g., **pst.ly/aZ9k2**). A simple, defensible approach: auto-increment an ID in the database, then encode that integer in base-62 (a-z, A-Z, 0-9) to produce a short string. This is technically LLD-adjacent detail, but HLD interviewers expect you to at least name the approach and its uniqueness guarantee, even without writing the encoding function.

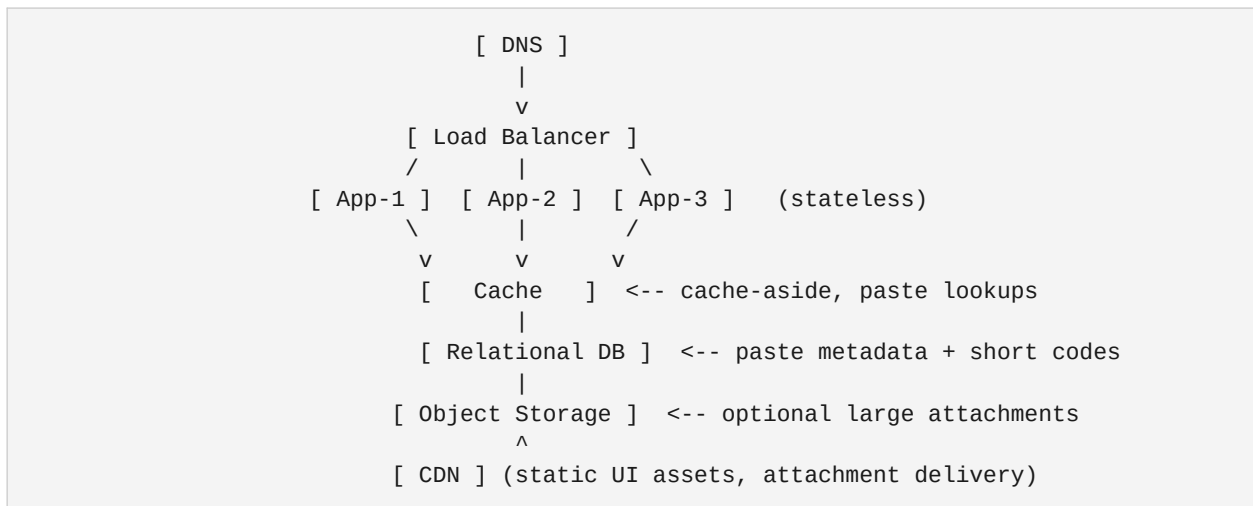
Step 5 — Sanity-check against non-functional requirements (Chapter 13)

Before calling the design done, hold it against the non-functional requirements from Step 1. Availability: three stateless app servers behind a load balancer means losing one server doesn't take the system down (Chapters 9, 12). Latency: cache-aside reads should comfortably hit low-millisecond P99s for cached pastes, though the very first read after a cache miss or expiry will be slower — worth stating explicitly rather than glossing over. This habit of checking the design against NFRs, not just functional requirements, is what Phase 4's interview framework will formalize into a repeatable final step.

2. Real-World Example / Case Study

This exact shape — write-light, read-heavy, cache-in-front-of-a-relational-store — describes a huge fraction of real consumer systems: Flipkart's product listing pages, Swiggy's restaurant menu pages, and PhonePe's merchant directory all share this profile. Mastering this pattern at Phase 1 depth is not a toy exercise; it's the single most reused architecture shape in real HLD interviews, which is exactly why "design a URL shortener" reappears as the very first full mock interview in Phase 4, Chapter 3.

3. Diagram



4. Trade-offs Table

Decision made	Alternative considered	Why this choice, at this scale
Relational DB for paste metadata	Document store	Simple, strict, small structured records; ACID not critical but no downside either
Cache-aside in front of DB	No cache	20:1 read:write ratio makes caching the single highest-leverage change
Single DB instance (no sharding yet)	Sharded DB from day one	~730GB/year is comfortably within one well-run instance's capacity
Base-62 encoded auto-increment ID	Random string + collision check	Simpler, no collision-retry logic needed, still short and unique

5. Common Interview Questions

Q1: "Walk me through your architecture for this system, end to end."

Model approach: Follow the same order used in this chapter — requirements, estimation, then architecture, narrating trade-offs at each step rather than just naming components. This ordering *is* the repeatable framework Phase 4, Chapter 1 formalizes.

Q2: "What's the very first thing that breaks if traffic grows 100x?"

Model approach: Reason from the estimation numbers — at 100x, write QPS goes from 10/sec to 1000/sec and yearly storage from ~730GB to ~73TB, at which point the single relational database becomes the bottleneck first (app servers scale horizontally easily; the database does not, without techniques Phase 2 covers). Naming the database as the first bottleneck, backed by your own numbers, is a strong answer.

6. Practice Exercises (Capstone)

- 1 Without looking back at this chapter, design this same pastebin system from scratch on paper: requirements, estimation, architecture diagram, and 2 trade-offs you made.
- 2 Now add one new requirement — pastes can be private and password-protected — and explain what changes in your design.
- 3 Identify, using your own estimation numbers, at what rough scale (QPS or storage) this design would start to strain, and name the single component you'd address first.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"System Design for Beginners: A Complete Roadmap"	Good sanity check that you've covered the right Phase 1 ground.
English	"Complete System Design Roadmap for 90 Days FAANG Interviews"	Shows where Phase 1 fits into a full interview-prep timeline.
Hindi	"Complete System Design Roadmap 2025 HLD & LLD" — CodeHelp	Full Hindi roadmap; useful to revisit now that Phase 1 vocabulary makes sense.
Hindi	"System Design Roadmap for Beginners" (Hindi & English resources)	Reinforces the same fundamentals-first structure this program follows.

8. Key Takeaways

- A repeatable order works for any HLD problem: requirements -> estimation -> architecture -> deep dive -> trade-offs.
- Back-of-the-envelope numbers should directly justify each architecture decision, not just decorate the answer.
- Read-heavy, write-light systems almost always benefit most from caching first, before anything more complex.
- Don't reach for sharding, multi-region, or microservices until your own numbers show you actually need them.
- This write-light/read-heavy/cache-in-front-of-relational-DB shape reappears across most real consumer products.

- You are now ready for Phase 2, which scales this exact design past the point where a single database and simple caching are enough.

Phase 1 complete

You've covered all 15 Phase 1 chapters. Redo this capstone exercise from memory before moving on. When you're ready, approve Phase 2 (Intermediate) to continue — it picks up exactly where this chapter's "what breaks at 100x" question leaves off.